

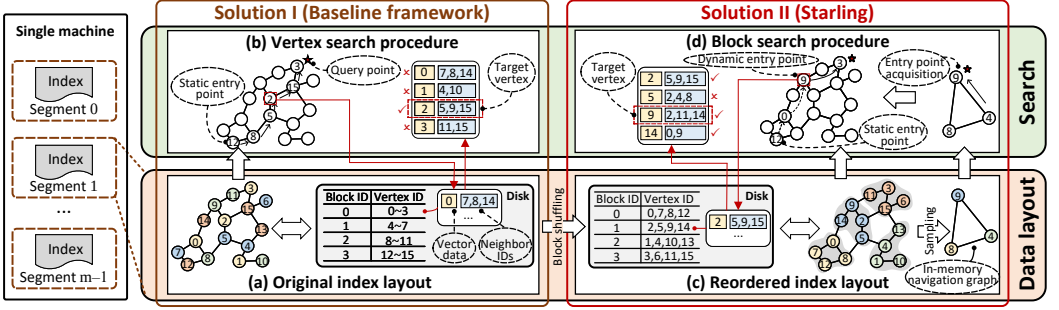


Fig. 2. Illustration of the data layouts and search strategies for the baseline and Starling, respectively.

searches on the disk-based graph, respectively. The total search time can be denoted as

$$T_{total} = T_{I/O} + T_{comp} + T_{other}. \quad (4)$$

In Eq. 4, $T_{I/O}$ emerges as the dominant factor influencing the search efficiency. For instance, upon visualizing the search time costs of DiskANN on BIGANN (Fig. 11(d)), it becomes evident that $T_{I/O}$ constitutes up to 92.5% of T_{total} , while T_{comp} and T_{other} collectively occupy less than 7.5%. This highlights the efficiency bottleneck as the cost of disk I/Os for HVSS on the disk-resident graph index. Therefore, our objective is to improve search efficiency by minimizing $T_{I/O}$. Note that we must not introduce additional space overhead (including memory and disk), compared to the original graph index, in order to adhere to the space capacity limitation of the data segment.

3 DESIGN PHILOSOPHY

We analyze two key factors that affect I/O time for existing graph index methods. Then we give an overview of Starling, illustrating its data layout and search strategy on the data segment.

3.1 I/O-efficiency Analysis

When conducting searches on the disk-based graph, I/O-efficiency depends on two primary factors: the vertex utilization ratio in each disk I/O and the length of the search path. The former indicates the extent to which useful vertices are loaded in a given block, while the latter denotes the number of hops required from the entry point to the result point. Both of these factors influence the number of disk I/Os and thus determine the I/O time. A higher vertex utilization ratio signifies that more relevant information is loaded in each disk I/O, leading to more effective utilization of disk bandwidth and requiring fewer disk I/Os for each query. A longer search path implies an increased number of disk I/Os during the search process. We delve into an analysis of the vertex utilization ratio and the search path length within the *baseline framework*² and reveal two underlying issues.

Problem 1: Poor data locality. Fig. 2(a) shows that the baseline assigns ID-consecutive vertices (a vertex contains the vector data and neighbor IDs) to the same blocks. For example, block 0 stores vertices 0~3. Since a block is the smallest disk I/O unit, reading vertex 2 also reads vertices 0, 1, and 3. This wastes disk bandwidth as the other three vertices are irrelevant to vertex 2. A naive way to avoid the wastage is to check all the vertices in a block. However, a segment has up to $10^7 \times$ more vertices than a block in a real-world scenario. The probability of finding a near vertex (w.r.t the target vertex) among the non-target vertices³ in the block is too low to improve efficiency effectively. Therefore, the baseline only checks the target vertex and discards the rest. This means a low vertex utilization ratio in each disk I/O. According to our evaluation on BIGANN (Tab. 2), up

²Unless specifically stated, we refer to DiskANN as the baseline framework.

³We load a block according to a target vertex. In this example, the target vertex is 2.

to 94% of data read from the disk is wasted. Obviously, ID-consecutive vertices in a block do not imply spatial proximity (e.g., vertices of the same color are scattered on the graph topology in Fig. 2(a)). Therefore, the baseline has poor data locality.

Problem 2: Long search path. The I/O complexity of searching on a disk-based graph index is proportional to the search path length. The baseline framework uses a fixed or random vertex as the entry point for the search. However, a segment may have tens of millions of vertices, so the entry point may be far from the query. In this case, only the last few vertices in the path are near the query and likely to be in the final result. However, we need to read each vertex along the path from the disk. Fig. 2(b) shows an extreme example where the entry point is the farthest vertex from the query point. It takes five hops to search from **12** to **3**. If we start from **9**, we can reduce the hops to two. For a dataset of tens of millions of vectors, even searching for only the top-10 nearest neighbors may generate a path of hundreds of hops to achieve high accuracy. Therefore, the baseline has a long search path.

3.2 Framework Overview

We present Starling, a framework designed to enhance the I/O-efficiency of disk-based graph index for HVSS on a data segment. While a single machine may encompass multiple segments, they all share the same index layout and search strategy. Therefore, our primary focus is on one specific segment. As depicted in Fig. 2(c), Starling preserves the graph's topology while optimizing the data layout to augment the vertex utilization ratio and diminish the search path length. Moreover, it employs a block search strategy aimed at reducing disk I/Os. Below, we provide an overview of the data layout and search strategy employed by Starling:

Data layout on disk. Starling improves data locality by shuffling the data blocks in accordance with the graph topology (Fig. 2(c) left). This aligns the data layout with the search procedure, which tends to visit neighboring vertices [68]. Specifically, Starling endeavors to store a vertex and its neighbors in the same block, enabling a single disk read to fetch not only the target vertex but also other likely candidates. This approach increases the vertex utilization ratio, as each disk I/O operation brings multiple relevant vertices. Furthermore, the search procedure can more readily jump to a closer vertex from the query, potentially reducing the number of required disk I/Os.

Data layout in memory. Starling identifies query-aware entry points for the disk-based graph using an in-memory navigation graph (Fig. 2(c) right). This approach incorporates the concept of multi-layered graphs [46, 49, 60] and involves sampling a small fraction of vectors from the disk-based graph to construct the navigation graph. During this process, any in-memory graph algorithm [24, 25] can be utilized. Given a query, Starling initially explores the in-memory navigation graph to obtain query-close vertices as entry points. Subsequently, it initiates the disk-based graph search from these identified points. This methodology effectively reduces the search path length on the disk-based graph.

Search strategy. Starling uses a block search strategy to exploit data locality (Fig. 2(d)). Unlike the baseline, which explores data on a vertex basis, this strategy processes data by blocks. This way, it benefits from the optimized data layout. For each loaded block, Starling computes the distance to the query for all vertices. Then, it selects the vertices that are close to the query and checks their neighbors for new search candidates. This strategy lowers the disk operation cost by exploring more relevant data per block but increases the computation cost. We further optimize the performance with three computation-specific optimizations (§5.1).

Example 1 shows how Starling reduces disk I/Os.

EXAMPLE 1. Fig. 2(b) shows a vertex search strategy that starts from vertex **12** and reaches the result vertex **3** in five hops for a given query point. This strategy needs at least six disk I/Os to

access the vertices in the path and their neighbors. This is inefficient, especially for large-scale scenarios, because the probability of finding a vertex close to the query among the non-target vertices in a loaded block is very low. Therefore, exploring data on a block basis with the original index layout is more harmful than helpful. In contrast, Starling reduces the disk I/Os to three with a reordered index layout, as shown in Fig. 2(d). The block search procedure works as follows: (1) It loads block 0 and visits vertices 12, 0, 7, and 8 in the block. Then it computes their distance to the query and selects 0 to visit its neighbors. (2) It loads block 1 with vertices {2, 5, 9, 14} and chooses 9 as the next hop. Vertex 3 as the search result will be found when Starling loads the next block according to vertex 11. With a navigation graph built on the vectors of {4, 8, 9} in memory, it obtains 9 as the entry point for disk-based graph search. Since 9 is close to the query, it only takes two disk I/Os (one to visit the entry point and one to visit its neighbors) to get the query result.

4 DATA LAYOUT

Fig. 2(c) shows how Starling organizes data on a segment. First, it builds a disk-based graph index on the full dataset and rearranges it by block shuffling to improve data locality. We can use different methods to construct Starling's disk-based graph, such as NSG [25], HNSW [49], and Vamana [35]. For more details on these methods, please refer to the original papers. We do not focus on developing a specific graph index algorithm since existing ones are well-studied but not suitable for disk deployment. Instead, we address the block shuffling problem on the disk-based graph (§4.1). Second, Starling samples some data points from the full dataset and builds an in-memory navigation graph (§4.2). This structure allows searching on the disk-based graph to begin from some query-aware entry points, which reduces the search path length.

4.1 Block Shuffling on the Disk

Notations. Let $G = (V, E)$ represent a disk-based graph index, where V and E denote the sets of vertices and edges, respectively. The Edges are directed and are stored as adjacency lists of vertices. Each vertex necessitates γ KB of storage to house its vector data, neighbor count λ , and a list of neighbor IDs (with a maximum length of Λ)⁴. The size of a disk block is η KB. Since we do not split the data of a vertex into two blocks, each block can accommodate at most $\varepsilon = \lfloor \eta/\gamma \rfloor$ vertices. Therefore, $\rho = \lceil |V|/\varepsilon \rceil$ blocks are required to store the graph index.

Next, we give the definition of block-level graph layout:

DEFINITION 1. Block-Level Graph Layout. *The block-level graph layout of a graph index is a scheme that assigns $|V|$ vertices to ρ blocks.*

EXAMPLE 2. Fig. 2(a) shows a graph index $G = (V, E)$ with 16 vertices ($|V| = 16$), four vertices per block ($\varepsilon = 4$), and four blocks in total ($\rho = 4$). For DiskANN [5] on 33 million BIGANN dataset ($|V| = 3.3 \times 10^7$) [3], each vector is 128-dimensional and one byte per value. If Λ is 31 and η is 4 KB, then $\gamma = (128 + 4 + 31 \times 4)/1,024$ KB (ID is unsigned integer type), $\varepsilon = 16$, and $\rho = 2,062,500$. Thus, DiskANN puts 16 ID-contiguous vertices in a block, such as 0 ~ 15.

Fig. 2(a) and (c) show two different ways of organizing the graph index on disk block level. As discussed in §3.1 (Problem 1), the graph layout affects disk I/Os during searching. Specifically, a layout without data locality lowers the vertex utilization ratio and increases random disk I/Os. We use the overlap ratio $OR(G)$ of the graph index G to measure the locality of the graph layout. For any vertex $u \in V$, $OR(G)$ is the average of $OR(u)$ over V , where $OR(u)$ is the proportion of vertices

⁴Each vertex is allocated Λ ID spaces for alignment, and padding is added when $\lambda < \Lambda$. For simplicity, the neighbor count λ is omitted in Fig. 2 and 3.

that are u 's neighbors among all the vertices except u in the block. We can compute $OR(u)$ by

$$OR(u) = \begin{cases} \frac{|B(u) \cap N(u)|}{|B(u)| - 1} & |B(u)| > 1 \\ 0 & |B(u)| \leq 1 \end{cases}, \quad (5)$$

where $B(u)$ is the set of vertices in the block that contains u and $N(u)$ is the set of u 's neighbors ($|N(u)| \leq \Delta$). In real-world datasets, the maximal vertex count ϵ is about 10 in each block and there is at most one block with a different ϵ (i.e., $|V|$ is not divisible by ϵ) in graph layout. We use $OR(B(u)) = \sum_{v \in B(u)} (OR(v)/|B(u)|)$ to denote the overlap ratio of the block $B(u)$. $OR(G)$ is calculated as $\sum_{u \in V} (OR(u)/|V|)$. A higher $OR(G)$ means better data locality and a higher vertex utilization ratio in a loaded block.

EXAMPLE 3. A graph layout with optimal data locality has $OR(G) = 1$, meaning that every vertex in a block is a neighbor of any other vertex in the block. However, we get $OR(G)$ close to 0 for the DiskANN [35] graph index on the 33 million BIGANN (Fig. 9(a)). This indicates the poor data locality of the DiskANN graph layout.

To enhance data locality, we use *block shuffling* to adjust the graph layout, which is defined as:

DEFINITION 2. *Block Shuffling.* Given a graph layout for a disk-based graph index G , the block shuffling aims to get a new layout that maximizes the $OR(G)$ while satisfying Def. 1.

In Def. 2, we aim to get a graph layout where every vertex in a block is a neighbor of other vertices within the same block. However, the graph index built on high-dimensional vectors is complex, as the neighborhood relationship encompasses both navigation and similarity aspects [24, 45]. A vertex may have neighbors that belong to different clusters [49], and all vertices exhibit a constant out-degree. Hence, it is challenging, if not impossible, to ensure that any two vertices in a block are mutual neighbors. We prove that the block shuffling problem is NP-hard in Theorem 4.1. Furthermore, it lacks a polynomial time approximation algorithm with a finite approximation factor. This motivates our block shuffling research, as the baseline framework merely populates a block with ID-contiguous vertices.

THEOREM 4.1. *The block shuffling problem is NP-hard and does not have a polynomial time approximation algorithm with a finite approximation factor unless $P=NP$.*

PROOF. (Sketch.) We reduce the block shuffling problem to the triple shuffling problem, which is strongly NP-complete [10, 59]. The triple shuffling problem is defined as follows: given $t = 3 \cdot \rho$ integers $\alpha_0, \alpha_1, \dots, \alpha_{t-1}$ and a threshold Ω such that $\Omega/4 < \alpha_i < \Omega/2$ and

$$\sum_{i=0}^{t-1} \alpha_i = \rho \cdot \Omega, \quad (6)$$

the task is to partition the numbers into ρ triples and DECIDE if these triples can be shuffled by swapping numbers between triples so that each triple sums up to Ω . We construct a graph G with cliques of size α_i for each integer α_i . Let Ω and ρ be the number of vertices in a block and the number of blocks in the graph layout of G , respectively. We demonstrate that the block shuffling problem on G has a solution iff the triple shuffling problem can be solved. Consequently, the block shuffling problem is NP-hard. Assuming the existence of a polynomial time approximation algorithm with a finite approximation factor for the block shuffling problem, we could use it to solve the triple shuffling problem. However, we establish that the triple shuffling problem cannot be solved by such an approximation algorithm with a finite approximation factor. This contradicts the assumption. (The detailed proof is available in Appendix A) $\square$

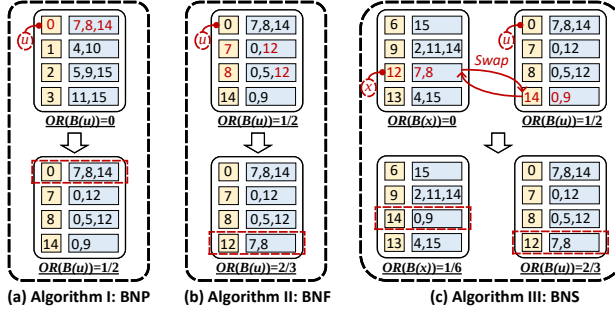


Fig. 3. Block shuffling. Refer to Fig. 2 for graph topology.

We design three heuristic algorithms to handle the shuffling problem. We first describe a straightforward solution (*Algorithm I*), and then present two optimized algorithms (*Algorithms II–III*).

Algorithm I: Block Neighbor Padding (BNP). This algorithm fills disk blocks in a one-by-one fashion. To fill blocks, it checks vertices in ascending order of IDs. If a vertex u is not assigned to any block yet, then the algorithm tries to assign the vertex and its neighbors to the current block. Once a block is full the algorithm opens a new block and assigns vertices to it. BNP has a time complexity of $O(|V|)$ and improves the overlap ratio $OR(G)$ by assigning vertices and their neighbors to the same block. However, the improvement is limited as some neighbors of u (denoted as $z, v \in N(u)$) may not be adjacent to each other, which lowers $OR(z)$ or $OR(v)$. Also, some neighbors of u may have been assigned to other blocks earlier, such as z also being a neighbor of o with a smaller ID than u . It cannot be stored with u as we store each vertex only once.

EXAMPLE 4. In Fig. 3(a), we have $B(u) = \{0, 1, 2, 3\}$ and $OR(B(u)) = 0$ for the original layout (the ID of u is 0). BNP puts 0 and its neighbors 7, 8, 14 in $B(u)$, so that $B(u) = \{0, 7, 8, 14\}$ and $OR(B(u)) = 1/2$. But vertex 12 cannot be in the same block with its neighbors 7 and 8, that is $OR(v) = 0$ for vertex v with ID 12.

Algorithm II: Block Neighbor Frequency (BNF). To further optimize $OR(G)$, we propose BNF, which aims to assign a vertex to the block that holds most of its neighbors, i.e., the block with the highest neighbor frequency. The BNF takes the result of BNP as an initial layout and optimizes $OR(G)$ iteratively. As illustrated in Algorithm 1: (1) It stores the current mapping of vertex IDs to block IDs and clears all the blocks of G (lines 3–5). (2) For each u in V , BNF tries to assign it to the blocks that hold its neighbors in the previous iteration (lines 6–14). The algorithm checks blocks in descending order of neighbor count and assigns u to the first block that is not full yet. If all the blocks are full, then BNF puts u in an empty block in $\mathcal{B}$ (lines 13–14). (3) It repeats this process until it reaches the iteration limit β or the $OR(G)$ gain between two iterations falls below a given threshold (lines 15–16). Then a new layout of G is returned. BNF has a time complexity of $O(\beta \cdot o \cdot |V|)$, where β is the number of iterations, o is the average out-degree, and $|V|$ is the number of vertices. This is because BNF needs to access the neighbors of all vertices in each iteration.

EXAMPLE 5. As shown in Fig. 3(b), BNF replaces 14 with 12 in $B(u)$, since 12 has two neighbors, 7 and 8, in the block. Thus, $B(u)$ becomes $\{0, 7, 8, 12\}$ and $OR(B(u)) = 2/3$.

Algorithm III: Block Neighbor Swap (BNS). The design of BNS is inspired by the NN-Descent method [22] in the literature of graph construction. BNS refines $OR(G)$ from an initial layout, which could be the result of BNP or BNF. In BNS, for a pair of vertices a and e that are neighbors of vertex u and belong to different blocks $B(a)$ and $B(e)$, BNS swaps vertices with the lowest overlap ratio (OR) in $B(a)$ and $B(e)$ to increase the sum of $OR(B(a))$ and $OR(B(e))$. Let o be the average

Algorithm 1: BLOCK SHUFFLING BY BNF

Input: block-level graph layout of $G = (V, E)$ from BNP, maximum iterations β , $OR(G)$ gain threshold τ

Output: new block-level graph layout of G

```

1  $\mathcal{B} = \{B_0, \dots, B_{\rho-1}\} \leftarrow$  all blocks ▷  $\rho$  is the number of blocks
2 while iterations  $\leq \beta$  do
3    $D \leftarrow$  mapping of vertex IDs to block IDs;
4   forall  $B_i \in \mathcal{B}$  do
5      $B_i \leftarrow \emptyset$ ; ▷ clear all blocks
6   forall  $u \in V$  do
7      $H \leftarrow \bigcup_{a \in N(u)} \{D(a)\}$ ; ▷ all neighbors' block IDs
8     while  $H \neq \emptyset$  do
9        $x \leftarrow$  block ID with the most neighbors in  $H$ ;
10      if  $B_x$  is not full then
11         $B_x \leftarrow B_x \cup \{u\}$ ; break;
12       $H = H \setminus \{x\}$ ; ▷ remove the block that is full
13      if  $H = \emptyset$  then
14         $\mathcal{B} \leftarrow \mathcal{B} \cup \{B_x\}$ ;
15      if  $OR(G)$  gain  $< \tau$  then
16        break;
17 return new layout of  $G$ 

```

out-degree, then the number of swaps is o^2 for each vertex in V . In each swap, the time complexity of computing one block's OR is $O(o \cdot \varepsilon)$, where ε is the number of vertices in a block. BNS operates in an iterative manner, where each iteration checks vertex pairs among the neighbor set of all vertices. Therefore, the time complexity of BNS is $O(\beta \cdot o^3 \cdot \varepsilon \cdot |V|)$, considering the number of iterations β . We prove that $OR(G)$ does not decrease with the number of iterations in Lemma 4.2.

EXAMPLE 6. In Fig. 3(c), BNS identifies the blocks $B(u)$ and $B(x)$ that contain vertices **0** and **12**, respectively, from the neighbors of vertex **7**. Then, it swaps vertices **12** and **14**, which have the lowest OR in their blocks and can increase $(OR(B(x)) + OR(B(u)))$ by swapping. Finally, we get $OR(B(x)) = 1/6$ and $OR(B(u)) = 2/3$.

LEMMA 4.2. *In BNS, the $OR(G)$ is a monotonically non-decreasing function of the number of iterations β .*

PROOF. In each iteration, an update is local and affects only two blocks' vertices. For any two neighbors a and e of vertex u ($B(a) \neq B(e)$), we prove that swapping vertices does not lower the sum of $OR(B(a))$ and $OR(B(e))$. Let $OR(B(a))_i$, $OR(B(e))_i$ and $OR(B(a))_j$, $OR(B(e))_j$ be the values of $OR(B(a))$, $OR(B(e))$ before and after an iteration, respectively. We swap two vertices in $B(a)$ and $B(e)$ only if $OR(B(a))_j + OR(B(e))_j > OR(B(a))_i + OR(B(e))_i$. Therefore, the sum of $OR(B(a))$ and $OR(B(e))$ is non-decreasing. $\square$

Analysis. Among our three block shuffling algorithms, BNP emerges as the fastest since it scans all vertices just once. On the other hand, BNF's efficiency is contingent on the number of iterations and does not ensure the convergence of $OR(G)$. However, BNF demonstrates proficiency, both in terms of efficiency and effectiveness, particularly on numerous real-world datasets. Meanwhile,

BNS, although possessing the highest time complexity, guarantees that $OR(G)$ does not decrease with iterations. In our implementation, we have parallelized BNF and BNS to enhance their speed. All three algorithms notably improve $OR(G)$ in contrast to the original graph layout, with BNS exhibiting the most significant improvement, followed by BNF and then BNP. In our evaluation (§6.5), higher $OR(G)$ improves vertex utilization ratio and search performance. In practice, the choice of algorithm can be tailored to specific requirements. Generally, we recommend BNF due to its adept balance between efficiency and effectiveness.

Time cost. We analyze the extra time cost caused by block shuffling. A recent study of current graph algorithms [68] shows that the graph index construction time complexity is always $O(|V| \log(|V|))$. The index construction involves high-dimensional vector distance calculation, which is very time-consuming. However, block shuffling only scans vertices and performs simple statistics, without any vector calculation. According to our evaluation, the block shuffling procedure introduces a relatively low additional time cost compared to the graph index construction process. For example, BNF only occupies 3%~10% of the graph index construction cost (see Fig. 8(a)).

Space cost. The disk-based graph index is stored as adjacency lists of vertices. Each vertex contains its vector data, neighbor count, and a list of neighbor IDs. Let Λ and D represent the maximum number of neighbor IDs and the vector dimensionality, respectively. The space complexity of the disk-based graph index is $O(|V| \cdot (D + \Lambda))$. In our experiments, we adjust Λ to conform to the disk capacity constraint of a segment. Note that the space cost of the disk-based graph index remains unchanged before and after block shuffling. This is because we only adjust the order of vertices and do not add any extra information.

Remarks. (1) Our block shuffling methods can work with any block size, not just the default 4KB for general disk. For example, we can extend to 8KB or 16KB blocks by modifying the block size. (2) Block shuffling is similar to but not identical to graph partitioning. Graph partitioning has been extensively studied for real-world graphs (e.g., Social Networks [55, 71]) in graph engines [48, 53, 63, 77], where the out-degree follows a power-law distribution (neighbors tend to cluster together) [8]. However, our graph index is based on high-dimensional vectors, where neighbors exhibit similarity and navigation traits (with about 50% long links [68]) and the out-degree distribution is uniform (neighbors may scatter across clusters) [68], making locality more challenging. Moreover, current graph partitioning methods thrive on real-world graphs with clustering properties but may falter in graph index for vectors. We evaluated some advanced graph partitioning methods for our block shuffling task, but they only gave limited improvement. For example, BNF shows 40% higher $OR(G)$ than an advanced graph partitioning method—KGGGP [57] for graph index built on the SSNPP dataset. Our block shuffling strategies are tailored for graph index with long navigation links [49] and prove well-suited under our problem setting. (3) Some recent works use some graph partitioning methods to reorder graph indexes [18, 34], but they only achieve limited improvement. In contrast, we design the block shuffling algorithms based on block-level graph layout on the disk, which leads to better disk-based HVSS performance.

4.2 In-Memory Navigation Graph

To reduce the search path length (ℓ), many in-memory graph-based algorithms use an additional structure, such as multi-level Voronoi diagrams [46]. The disk I/O cost depends directly on ℓ for searching on the disk-based graph index, so a shorter ℓ is even more crucial.

Our in-memory navigation graph reduces ℓ by finding better entry points for the disk-based graph search. To obtain such a graph, we employ two steps: (1) *sample data points* and (2) *build a graph index*. First, we randomly sample some data points from all the data in a segment, based on the memory limit of a segment. Second, we use the same algorithm as the disk-based graph (such

as HNSW [49], NSG [25]) to build a navigation graph on the sampled data. The navigation graph is memory-resident and can quickly return the dynamic entry points that are close to a query.

Time cost. Starling builds an in-memory graph only on a small data subset V' , lowering the time complexity to $O(|V'| \log(|V'|))$, where $|V'|$ is less than 10% of the total number of vertices in a segment. This process is notably faster than constructing the graph on the entire dataset. In our evaluation, the in-memory graph construction demonstrates a low time cost (e.g., only 5.5% of the total index processing time in Fig. 8(a)).

Space cost. The in-memory graph and the disk-based graph share the same storage format. Let Λ' and D denote the maximum number of neighbor IDs and the vector dimensionality, respectively. The space complexity of the in-memory graph is $O(|V'| \cdot (D + \Lambda'))$. In our implementation, we adjust $|V'|$ and Λ' to adhere to the memory limitation of a segment.

5 SEARCH STRATEGY

The search strategy of Starling consists of two components: (1) *vertex search on the in-memory navigation graph* and (2) *block search on the disk-resident graph*. The first component is designed to quickly navigate to the query's neighborhood without requiring disk access. It employs the same vertex search strategy as existing graph algorithms. The vertex search results serve as the entry points for the second component. To exploit the improved data locality, we utilize block search to explore not only the target vertex but also other vertices in the same block for each disk I/O. Next, we will outline the fundamental block search strategy and its optimizations. Following that, we will show how we build the ANNS and RS algorithms based on the block search.

5.1 Block Search

Starling offline assigns vertices and their neighbors to the same block to improve data locality. This way, one online block read from disk provides multiple relevant vertices. The block search strategy efficiently explores all the useful data in a block. It updates the current search results by calculating the distance from each vertex in the block to the query. It also checks the neighbor IDs of the closer vertices for new search candidates. Starling further enhances this strategy with three computation-specific optimizations.

Block pruning. We aim for a graph layout with $OR(G) = 1$, meaning every vertex accessed by the block search is necessary. However, this ideal layout is very hard or impossible to achieve for a graph index built on high-dimensional vectors. Usually, our block shuffling results in $OR(G)$ ranging from 0.3 to 0.6 on many real-world datasets (Fig. 9(a)). This implies that some vertices in a block are irrelevant. To avoid exploring irrelevant data, Starling prunes each loaded block. Specifically, it sorts the vertices in a block by their distance to the query vector in ascending order. Then, it only checks the neighbor IDs of the top- $((\epsilon - 1) \cdot \sigma)$ vertices for new search candidates, where $(\epsilon - 1)$ is the number of vertices excluding the target vertex in a block and σ is pruning ratio ($0 < \sigma \leq 1$). This way, the neighbor IDs of distant vertices are discarded early. In our evaluation, $\sigma = 0.3$ is always the optimal value for better performance.

I/O and computation pipeline. In the block search stage, disk read (DR) and distance computation (DC) are two main operations. They occupy about 80% of the block search time (see Fig. 11(d)). A naive implementation is to execute DR and DC serially—that is—load a block by DR and then select the next hop by DC (as done in DiskANN [35]). However, this schedule is inefficient because DR and DC are idle alternately. This wastes disk bandwidth and CPU. To avoid this, Starling uses an I/O and computation pipeline that performs DR and DC concurrently (cf. Algorithm 2). Specifically, Starling first executes DC for the target vertex u in the current loaded block (lines 6–7). Then, it immediately conducts the next DR, while also performing DC of other vertices in the block that

contains u (line 11). Although each current DR decision does not consider the non-target vertices from the previous DR, it is acceptable as Starling fully utilizes the disk bandwidth and CPU. We will show in §6.5, that I/O and computation pipeline reduces query latency significantly.

PQ-based approximate distance. Recall that we need to load the full-precision vectors of all neighbors of the visited vertex to determine the next hop. Block shuffling mitigates such disk access by filling a block with many neighboring vertices. However, the number of neighbor IDs of each vertex is usually several times the number of vertices in a block, so many neighbors still require extra disk I/Os. To address this issue, we use approximate distance instead of exact distance with full-precision vectors. This is based on the observation that the next hop decision can be made by approximate computation with little accuracy damage [15]. Specifically, Starling preprocesses the full dataset by PQ [36] (like DiskANN [35]), a popular compression method. It encodes the full-precision vectors into short codes that reside in the main memory. Indeed, Starling efficiently obtains the approximate distances between the neighbors (whose full-precision vectors are not in memory) and the query by the short codes without disk I/Os to make the next disk read decision.

Time cost analysis. Let ξ be the vertex utilization ratio and ε be the number of vertices in a disk block. Then $\xi \cdot \varepsilon (\geq 1)$ represents the number of vertices accessed in each disk I/O. The vertex access complexity of the graph index is $O(o \cdot \ell)$ [68], where ℓ is the search path length and o is the average out-degree of visited vertices. Therefore, the I/O time complexity of searching on the disk-based graph is $O((o \cdot \ell)/(\xi \cdot \varepsilon))$. In addition, the computation-specific block search optimizations further enhance search efficiency.

5.2 Approximate Nearest Neighbor Search

The ANNS strategy in Starling uses two ordered lists to store the candidates and search results: the candidate set and the result set. It starts the search by adding the entry points of the disk-based graph to the candidate set. These entry points are obtained on the in-memory navigation graph. Then it selects the unvisited vertex that is closest to the query from the candidate set and performs a block search. It updates both sets with the block search outcome. The procedure ends when all the vertices in the candidate set are visited. To improve efficiency, the strategy limits the size of the candidate set to prevent too many candidate vertices. The result set has no size limit because it is sorted only when the search terminates.

Starling performs ANNS for a query q as follows (cf. Algorithm 2). (1) It finds the entry points S of q on the in-memory navigation graph G_m (line 1). (2) It initializes a fixed-size candidate set C and a result set R with S (lines 2–3). It sorts C by approximate distance using PQ short codes. (3) It picks the top-1 unvisited vertex u in C as the target vertex and reads the block B containing u from the disk (lines 4–5). (4) It adds u 's neighbor IDs to C and u to R (lines 6–7). (5) It prunes some vertices that are far from q from B , adding the closer vertices into B' (line 8). (6) It executes (3) for the next top-1 unvisited vertex v and (4) for the vertices in B' in parallel, then it executes (4)–(5) for v (lines 10–12). (7) It terminates when C has no unvisited vertices. Finally, it sorts R by exact distance using full-precision vectors and returns top- k results in R .

5.3 Range Search

A range search (RS) query returns all the vectors within a search radius r . The result length depends on the vector distribution and can vary a lot among queries. For a dataset with tens of millions of vectors, the same r may give zero to thousands of results. So, the RS algorithm should handle different result lengths. A simple RS strategy is to do ANNS repeatedly with different k values to check the result length. However, this is inefficient because it will revisit the same vertices multiple times, causing extra computation and disk I/Os.

Algorithm 2: ANNS

Input: in-memory navigation graph G_m , PQ short codes, disk-based graph G_d , query q , candidate set C with fixed size, result set R , pruning ratio σ

Output: top- k results of q

```

1  $S \leftarrow$  entry points of  $q$  by a vertex search strategy on  $G_m$ ;
2  $C \leftarrow S$ ;                                ▶ sort by PQ distance to  $q$ 
3  $R \leftarrow S$ ;                                ▶ compute exact distance to  $q$ 
4  $u \leftarrow$  top-1 unvisited vertex in  $C$ ;        ▶ target vertex
5  $B \leftarrow$  load the block including  $u$  from  $G_d$ ;    ▶ DR
6 update  $C$  according to  $u$ 's neighbor IDs;        ▶ DC
7 add  $u$  to  $R$  based on  $u$ 's full-precision vector;    ▶ DC
8  $B' \leftarrow$  top- $((\epsilon - 1) \cdot \sigma)$  vertices in  $B \setminus \{u\}$ ;    ▶ block pruning
9 while  $C$  has unvisited vertex do
10    $v \leftarrow$  top-1 unvisited vertex in  $C$ ;        ▶ PQ-based routing
11   conduct line 5 for  $v$ , and lines 6–7 for the vertices in  $B'$  in parallel;
12   ▶ I/O and computation pipeline
13   conduct lines 6–8 for  $v$  and the block including it;
14 return top- $k$  vertices in  $R$                     ▶ sort by exact distance to  $q$ 

```

Starling performs RS for a query q based on block search. It uses a candidate set C , a result set R , and a kicked set P to store candidate vertices, results, and vertices kicked out from C , respectively. Starling changes the length limit of C dynamically to handle different result lengths. The steps of RS are as follows. (1) It obtains the entry points from the in-memory navigation graph and initializes C and R with them. (2) It iteratively explores C and updates C and R (like ANNS). It also adds the unvisited vertices kicked out from C to P . (3) When all the vertices in C are visited, it calculates the ratio of R 's length to C 's length. Given a ratio threshold φ , if

$$\frac{|R|}{|C|} \geq \varphi, \quad (7)$$

it doubles the size of C and restarts the search. This is because a high ratio means that most candidates are results. In this case, exploring more vertices may find new results. (4) In the next search, Starling adds the closer vertices (w.r.t q) in P to C and repeats (2)–(3) for unvisited candidates in C . (5) The search stops when Eq. 7 is not met. After changing the length of C , it resumes the search with the previous results in R , candidates in C , and some closer vertices (w.r.t q) in P . This avoids extra computation and disk access. Our evaluation shows that $\varphi = 0.5$ is optimal.

6 EXPERIMENTS

We evaluate the following aspects of Starling: (1) search performance (§6.2), (2) I/O-efficiency (§6.3), (3) index cost (§6.4), (4) ablation study (§6.5), (5) parameter sensitivity (§6.6), (6) scalability (§6.7), (7) query distribution (§6.8), (8) segment setup (§6.9), (9) large-scale search results (§6.10), and (10) billion-scale data (§6.11). Our source code and datasets are available at: <https://github.com/zilliztech/starling>. Kindly refer to Appendix for additional evaluations.

6.1 Experimental Setting

Datasets. We use four public real-world datasets [3] that vary in data type, dimensions, distance, and query type. Tab. 1 shows their details. Unless specified, we limit the raw vectors per segment